

Trabajo Práctico Integrador

*Virtualización con VirtualBox: Implementación de un Entorno Aislado (Sandbox)
para Ejecución de Scripts con lenguaje Python*

Título del trabajo:	Virtualización con VirtualBox: Implementación de un Entorno Aislado (Sandbox) para Ejecución de Scripts con lenguaje Python
Alumno/a 1:	Gerardo R. Ocampo (Comisión 20) – gerocampo1993@gmail.com
Alumno/a 2:	Misseri Nicolas (Comisión 5)– nicomisseri1@gmail.com
Materia:	Arquitectura y Sistemas Operativos
Profesor/a:	Tutor Comisión 5 - Octavio Rolón Tutor Comisión 20- Jonathan Zarate
Fecha de entrega:	21/06/2026
Enlace al video explicativo:	https://drive.google.com/drive/folders/1yErIRiEg2ji6JxtY7zxcHJUrRPNspo5w?usp=drive_link

Índice

Índice	1
1. Introducción	2
2. Marco Teórico	2

3. Caso Práctico	5
4. Metodología Utilizada	5
5. Resultados Obtenidos	5
6. Conclusiones	5
7. Bibliografía	5
8. Anexos	6

1. Introducción

El presente trabajo aborda la **virtualización** como una competencia valiosa y estratégica en la industria tecnológica actual. Esta temática fue seleccionada debido a su gran utilidad en el desarrollo de software y la administración de sistemas, al permitir la **experimentación en ambientes seguros y controlados** por medio del concepto de “**Sandbox**” o entorno aislado. Particularmente, esta herramienta facilita la ejecución de un sistema operativo dentro de otro sin comprometer la integridad ni los datos del sistema operativo principal (normalmente Windows).

Los objetivos principales de este proyecto consisten en la implementación práctica de una distribución Linux (específicamente Ubuntu) en **Virtual Box**. A través de este proceso, se busca alcanzar una comprensión profunda de los mecanismos de simulación de hardware y la gestión de recursos críticos. El trabajo se centra en la instalación, configuración y optimización **de una máquina virtual (MV)**, analizando los parámetros esenciales para asegurar la estabilidad, la funcionalidad y el uso eficiente de la CPU y la memoria RAM asignada.

Asimismo, el proyecto integra fundamentos de programación de nivel inicial mediante el desarrollo de un **script en Python**. Dicho código es gestionado a través de un repositorio colaborativo en **GitHub**, lo que fomenta el uso de herramientas de control de versiones y , al mismo tiempo, asegura una documentación técnica organizada. La ejecución de este script dentro de la MV permite validar la operatividad del entorno virtualizado de forma aislada

Finalmente, se espera que el desarrollo de este proyecto permita adquirir las destrezas necesarias para el dominio de los entornos virtuales descritos, con el fin de simular escenarios de servidores reales y más complejos en el futuro ejecución profesional de la programación.

2. Marco Teórico

La principal referencia y sustento teórico de este trabajo son los materiales bibliográficos y recursos aportados por la cátedra para el módulo 7 de la materia (a excepción de Docker, no descrito en el presente trabajo).

El fundamento de este proyecto reside en la comprensión de la arquitectura de sistemas y las tecnologías de abstracción de hardware.

2.1. Concepto de Virtualización y Máquina Virtual (MV)

La virtualización se define como una tecnología que permite crear versiones virtuales de recursos informáticos, tales como sistemas operativos, servidores o redes.

Este proceso se logra mediante un software que abstrae los recursos físicos y los presenta como entornos independientes. Una Máquina Virtual (MV) es una simulación de hardware y sistemas completos que funciona como una computadora independiente.

Un concepto crítico en esta práctica es el de entorno aislado o "*Sandbox*" (caja de arena). Este mecanismo asegura que cualquier proceso o fallo dentro de la MV no afecte la integridad del sistema operativo anfitrión (host) ni los datos del equipo físico, proporcionando un ambiente seguro para el desarrollo y las pruebas de software.

2.2. El Hipervisor: Clasificación y Función

El componente central que gestiona las máquinas virtuales es el hipervisor o monitor de máquina virtual. Su función es actuar como intermediario entre el hardware físico y los sistemas operativos invitados (guests), asignando recursos como CPU, memoria, disco y red.

Se clasifican en dos categorías principales:

Tipo 1 (Nativo o Bare-metal): Se ejecuta directamente sobre el hardware físico (ej. VMware ESXi, Hyper-V).

Tipo 2 (Alojado o Hosted): Se instala como una aplicación sobre un sistema operativo anfitrión preexistente. **Oracle VM VirtualBox, la herramienta utilizada en este trabajo**, pertenece a esta categoría, destacándose por su facilidad de uso en entornos de pruebas.

2.3. Mecanismos de Virtualización de Recursos

Para que una MV opere con fluidez, el hipervisor debe emular los componentes físicos (Tabla 1):

Tabla 1. Componentes físicos que requieren emulación

Componentes a emular	Descripción
CPU	Se utilizan técnicas como la traducción binaria, donde el hipervisor intercepta instrucciones privilegiadas de la MV para redirigirlas de forma segura al anfitrión. Los procesadores modernos incluyen soporte por hardware (Intel VT-x o AMD-V) que mejora la eficiencia al crear contextos aislados directamente en el procesador
Memoria	Se emplean tablas de páginas sombreadas (<i>Shadow Page Tables</i>) para mapear direcciones virtuales de la MV a las físicas del anfitrión, garantizando que una MV no acceda a la memoria de otra.
Almacenamiento	El hipervisor crea discos virtuales, que son archivos contenedores (ej. formato .vdi en VirtualBox) que el sistema invitado percibe como una unidad de disco duro física.

2.4. Oracle VM VirtualBox: Características y Justificación de Uso

Oracle VM VirtualBox se define como un hipervisor de tipo 2 o alojado que permite la creación y gestión de máquinas virtuales (VM), actuando como un intermediario entre el hardware físico y

los sistemas operativos invitados. De acuerdo con la documentación oficial, este software simplifica las operaciones de TI al reducir los costos operativos, ya que permite consolidar múltiples configuraciones de escritorio y servidor en una sola plataforma física (**Oracle Corporation, 2026**).

Tabla 2. Características principales de VirtualBox

Versatilidad de Plataforma	Es ejecutable en cualquier escritorio con arquitectura x86, siendo compatible con sistemas operativos host como Windows, GNU/Linux y macOS.
Gestión de Recursos Escalable	Permite a los equipos de desarrollo consolidar cargas de trabajo masivas, admitiendo hasta 32 CPUs virtuales (vCPU) por cada instancia
Facilidad de Adopción	Provee una interfaz gráfica de usuario (GUI) intuitiva junto con una potente interfaz de línea de comandos (CLI), facilitando la transición y el uso para desarrolladores y administradores.
Entornos de Prueba Seguros	Acelera los procesos de garantía de calidad y pruebas mediante la implementación de entornos aislados o "Sandbox", lo que permite experimentar con software nuevo sin

	comprometer la integridad del sistema anfitrión.
Automatización y Seguridad	Facilita las implementaciones en la nube y permite que los trabajadores remotos accedan a aplicaciones de forma segura, garantizando la continuidad operativa.

La elección de esta herramienta para el presente trabajo se justifica por su naturaleza multiplataforma y su gratuidad, factores que la convierten en un estándar accesible para el aprendizaje de la administración de sistemas y el despliegue de soluciones virtualizadas de alto rendimiento.

2.5. Entorno de Ejecución: GNU/Linux (Ubuntu)

El sistema operativo invitado seleccionado es Ubuntu, una distribución basada en el kernel de Linux.

Linux se caracteriza por una estructura de archivos jerárquica que nace de un único directorio raíz (/). A diferencia de Windows, Linux es un sistema case-sensitive (distingue mayúsculas de minúsculas) y basa su configuración en la edición de archivos de texto plano, ofreciendo mayor flexibilidad y control técnico

A fin de ilustrar las diferencias operativas entre los sistemas comparados, se presenta la siguiente tabla:

Tabla 3. Comparación de características principales de Windows vs GNU/Linux (Ubuntu)

Característica	Windows	GNU/ Linux (Ubuntu)
Punto de Origen	Unidades lógicas	Jerarquía de directorios única

	identificadas con letras (C:, D:)	que nace de la raíz (/)
Separador de Rutas	Utiliza la barra invertida o backslash (\)	Utiliza la barra de división o forward slash (/)
Sensibilidad (Case-sensitive)	No distingue entre mayúsculas y minúsculas (Case-insensitive)	Distingue estrictamente entre mayúsculas y minúsculas (Case-sensitive)
Gestión de Configuración	Se basa en herramientas gráficas y un Registro centralizado binario	Se basa habitualmente en la edición de archivos de texto plano (directorio /etc)
Extensiones de Archivo	Cruciales para que el sistema identifique el tipo de archivo y con qué abrirlo	Menos importantes; el sistema utiliza métodos como el comando file para identificar el contenido

2.6. El estándar de Jerarquía del Sistema de Archivos (FHS) - Linux

En los sistemas operativos basados en Linux, la organización de los datos no es arbitraria, sino que sigue el estándar FHS (Filesystem Hierarchy Standard).

La presente norma técnica tiene como objetivo primordial mantener la consistencia y uniformidad entre las diversas distribuciones, permitiendo que tanto los usuarios como las aplicaciones puedan predecir la ubicación exacta de los archivos y bibliotecas

En este contexto, las partes de un mismo software se distribuyen en diferentes directorios según su naturaleza: **los ejecutables en un sector, las configuraciones en otro y los datos variables en uno distinto.**

La estructura lógica definida por el FHS se representa como un árbol invertido que se origina en un único punto de partida denominado directorio raíz (/). Dentro de esta jerarquía, el estándar asigna propósitos específicos a directorios fundamentales, entre los que se destacan:

Tabla 4. Directorios fundamentales

/bin	Contiene los comandos binarios esenciales del sistema que deben estar disponibles para todos los usuarios
/etc	Aloja los archivos de configuración tanto del sistema operativo como de las aplicaciones instaladas
/home	Directorio destinado a los espacios personales y documentos de los usuarios estándar
/var	Almacena archivos de contenido variable que cambian durante el funcionamiento normal, tales como los registros de eventos (logs) y bases de datos
/dev	Representa los dispositivos de hardware como archivos especiales, permitiendo al sistema interactuar con ellos mediante comandos de lectura y escritura

Mantener esta organización no solo facilita la navegación y el mantenimiento técnico, sino que constituye el pilar de la seguridad y estabilidad en Linux, al permitir una gestión estricta y segregada de los permisos de acceso sobre cada rama del árbol de directorio.

A modo de cierre del marco teórico, la arquitectura de virtualización analizada permite establecer una infraestructura de abstracción que redefine la interacción entre el software y los componentes físicos de una computadora. La implementación de máquinas virtuales mediante hipervisores de tipo 2, como Oracle VM VirtualBox, garantiza la creación de entornos de trabajo que operan bajo el principio de aislamiento (sandbox), asegurando la integridad del sistema anfitrión mientras se ejecutan procesos en sistemas operativos con estructuras lógicas distintas,

como el estándar FHS de Linux. Este marco conceptual proporciona las bases necesarias para abordar la resolución del problema técnico planteado en el siguiente apartado del informe.

3. Caso Práctico

La práctica realizada en el Trabajo Práctico Integrador consistió en la implementación técnica de un entorno aislado (Sandbox) mediante la creación de una Máquina Virtual (MV) para la ejecución segura de scripts en Python.

A continuación, se describen las etapas que conformaron el desarrollo del caso práctico:

1. Preparación del Sistema Anfitrión (Host)

Primero que todo, se verificó que el hardware del equipo físico (procesador Intel i3-9100) tuviera habilitada la virtualización por hardware (Figura 1). Esta comprobación se realizó a través del Administrador de tareas en Windows y, en las instancias necesarias, mediante la configuración de la BIOS/UEFI para activar las instrucciones especiales del CPU necesarias para un rendimiento óptimo de las MVs.

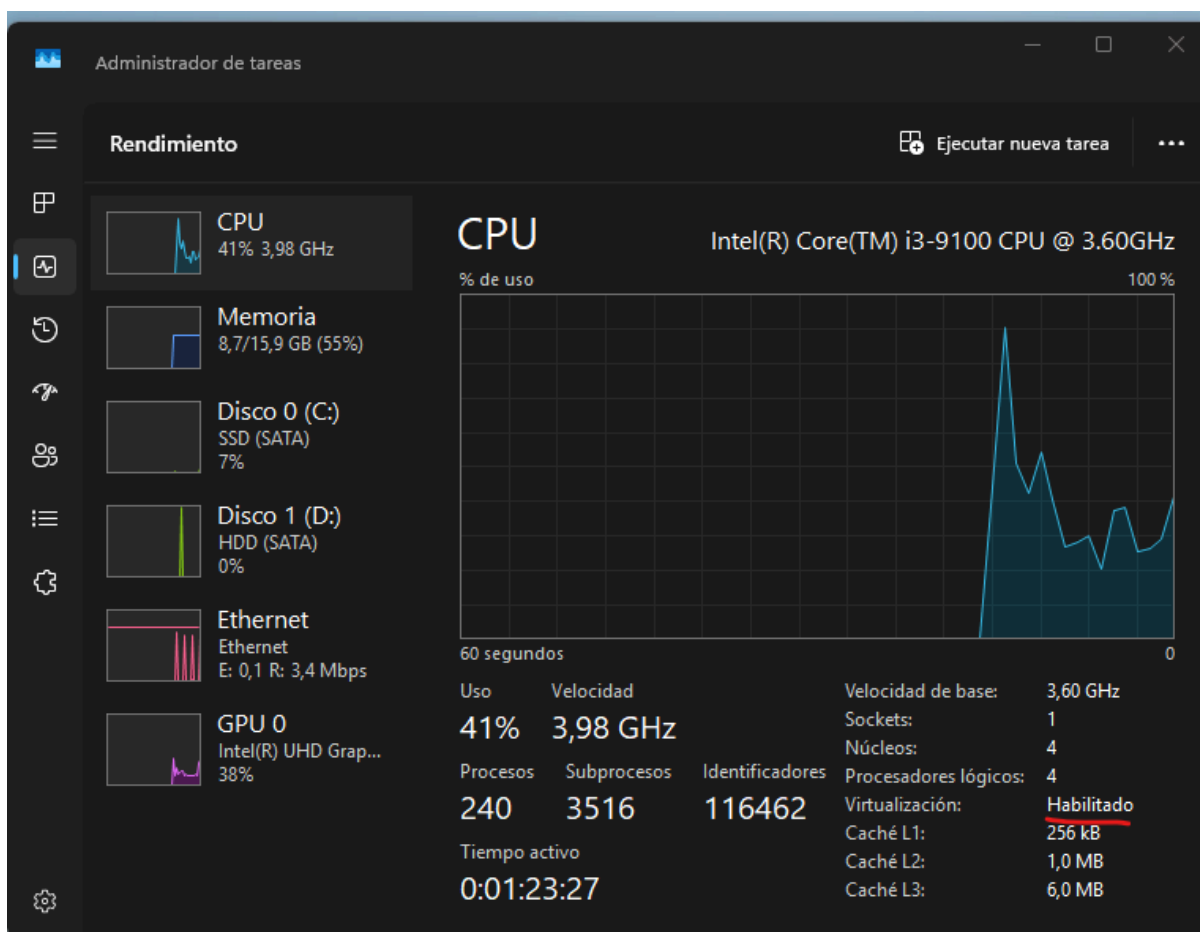


Figura 1. Captura del administrador de tareas con las características de la PC utilizada (máquina anfitriona). La línea roja resaltada indica que la virtualización se encuentra habilitada.

Tal como se observa en la figura 1, la línea resaltada en rojo “Virtualización: Habilitada” revela que el procesador (CPU) utilizado y la BIOS permiten ejecutar máquinas virtuales, y que esa función está activada actualmente.

2. Descarga e Instalación de VirtualBox

A continuación, se procedió a **descargar e instalar Virtualbox desde <https://www.virtualbox.org>**, siguiendo el proceso de instalación standard.

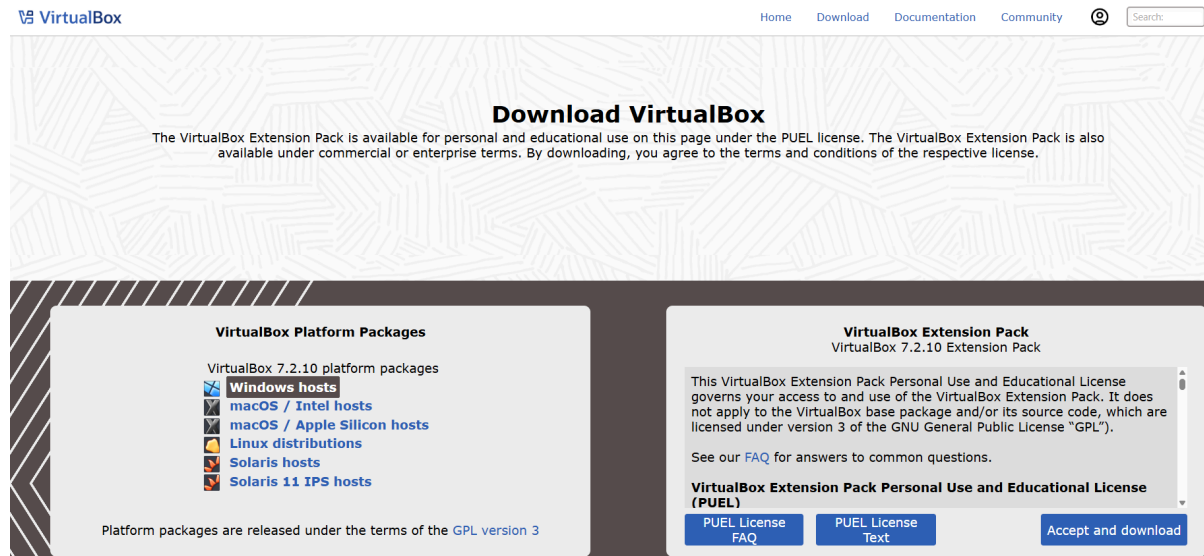


Figura 2. Captura del sitio Web de VirtualBox, sección *Download*.

Esta herramienta nos permitió gestionar los recursos físicos del anfitrión para asignarlos de manera lógica a la máquina virtual.

3. Implementación de la Máquina Virtual (Ubuntu)

Previo a la implementación de la máquina virtual se descargó Ubuntu Desktop (versión 25.04) del sitio web oficial (Figura 3).

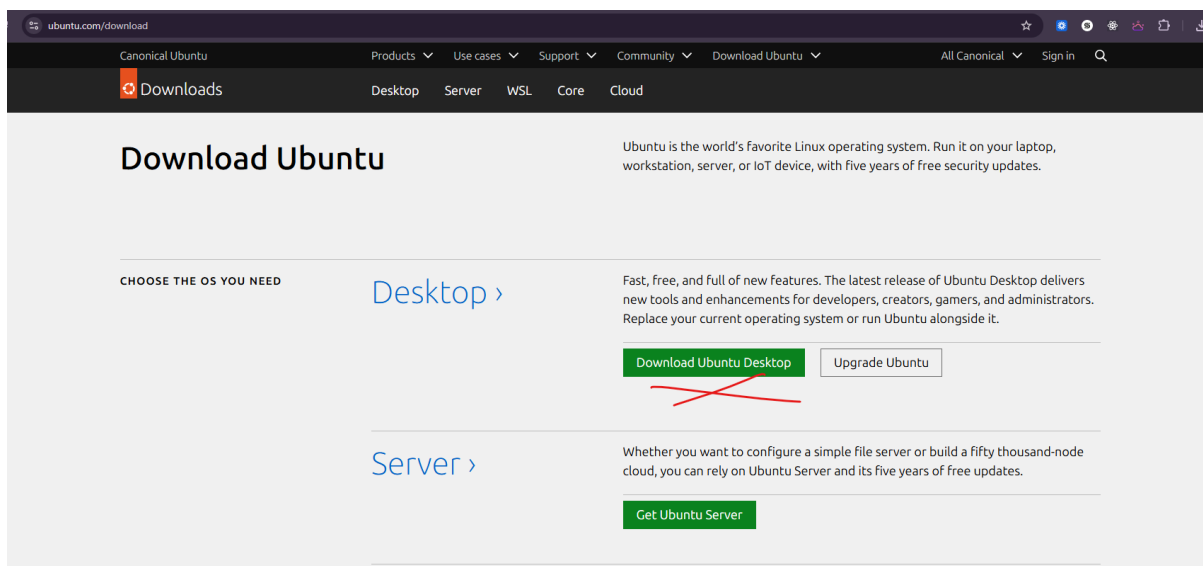


Figura 3. Captura del sitio Web de Linux (Ubuntu), sección *Download*.

A continuación, se utilizó Virtual Box y se procedió a montar la imagen ISO descargada de Ubuntu (Figura 4).

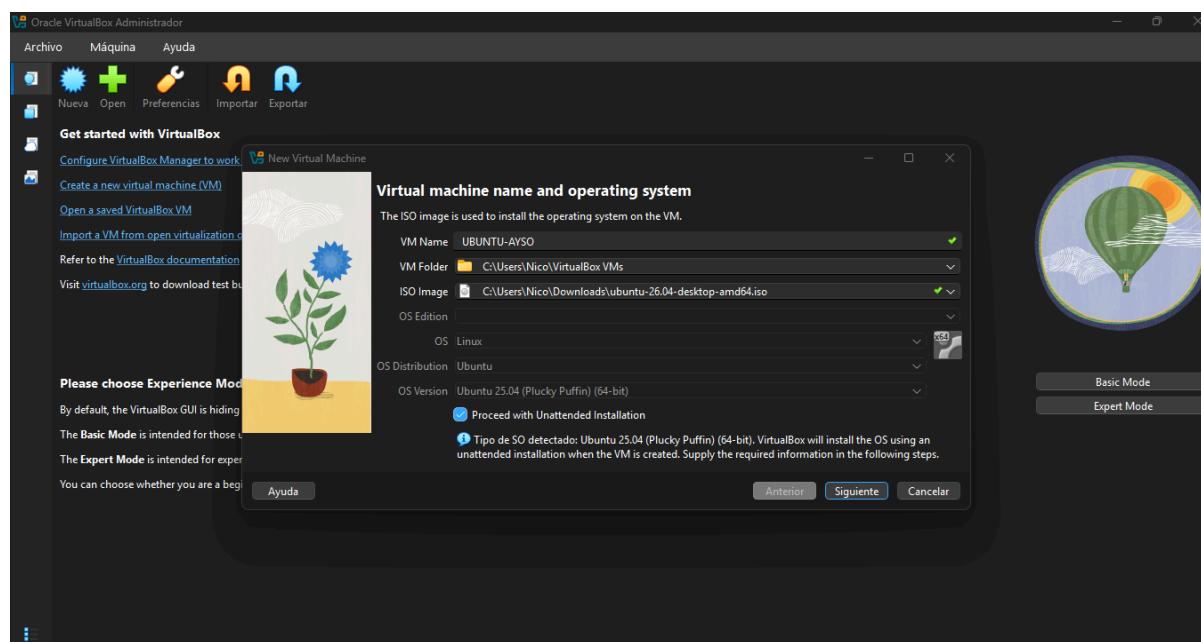


Figura 4. Captura del programa VirtualBox en ejecución.

Se creó una instancia virtual denominada "UBUNTU-AYSO" con la siguiente configuración de recursos basada en los requerimientos del sistema operativo invitado:

Tabla 5. Características implementadas en la MV.

Imagen ISO	Se utilizó la distribución Ubuntu Desktop (versión 25.04)
Memoria RAM	Se asignaron 3072 MB (aprox. 3 GB) para garantizar la fluidez del entorno gráfico
Procesador	Se destinaron 2 núcleos de CPU virtuales
Almacenamiento	Se configuró un disco duro virtual de 15 GB

Red	Se estableció una conexión en modo NAT para otorgar conectividad a Internet a la MV
------------	---

Esta configuración, se cargó en VirtualBox luego de especificar Usuarios y Contraseña. En la siguiente figura se observa una captura con la antes configuración descrita:

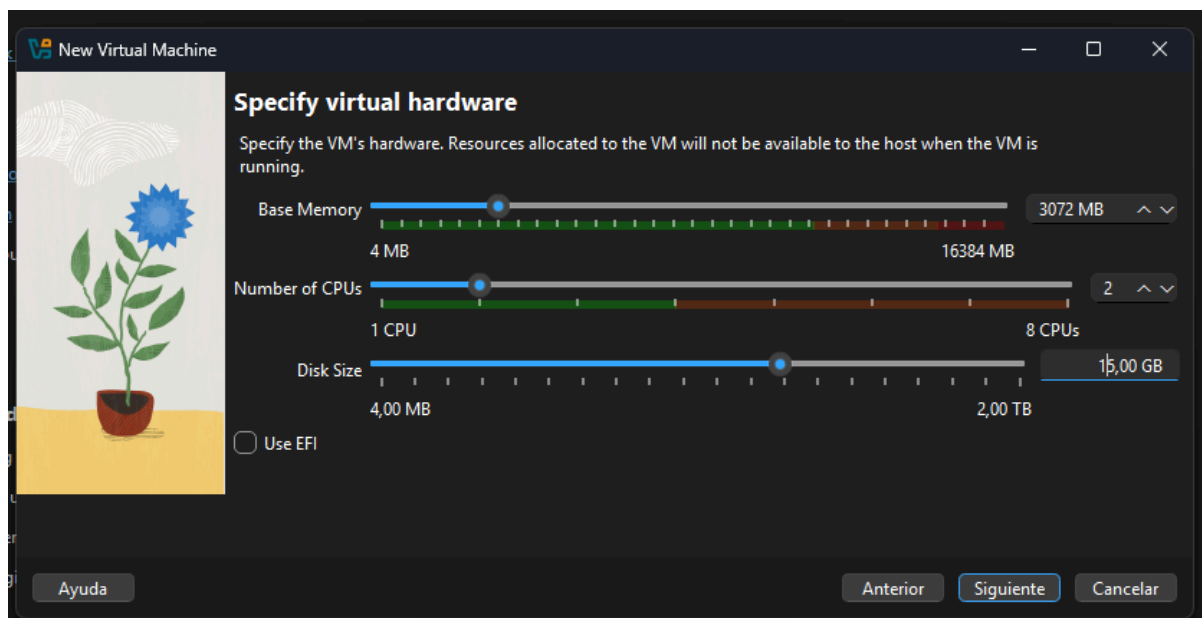


Figura 5. Captura de la configuración de la nueva máquina virtual en VirtualBox.

En la figura 6 se observa una captura del proceso de instalación automático de UBUNTU en la máquina virtual, luego en la figura 7 se observa el proceso finalizado.

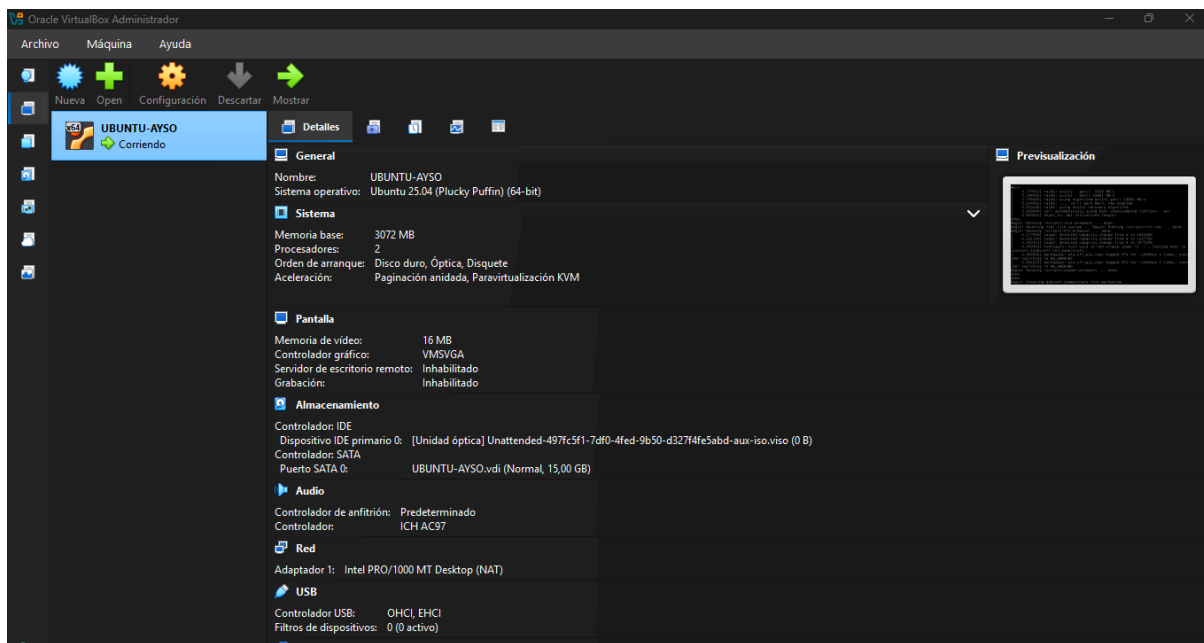


Figura 6. Instalación de Ubuntu en la nueva máquina virtual.

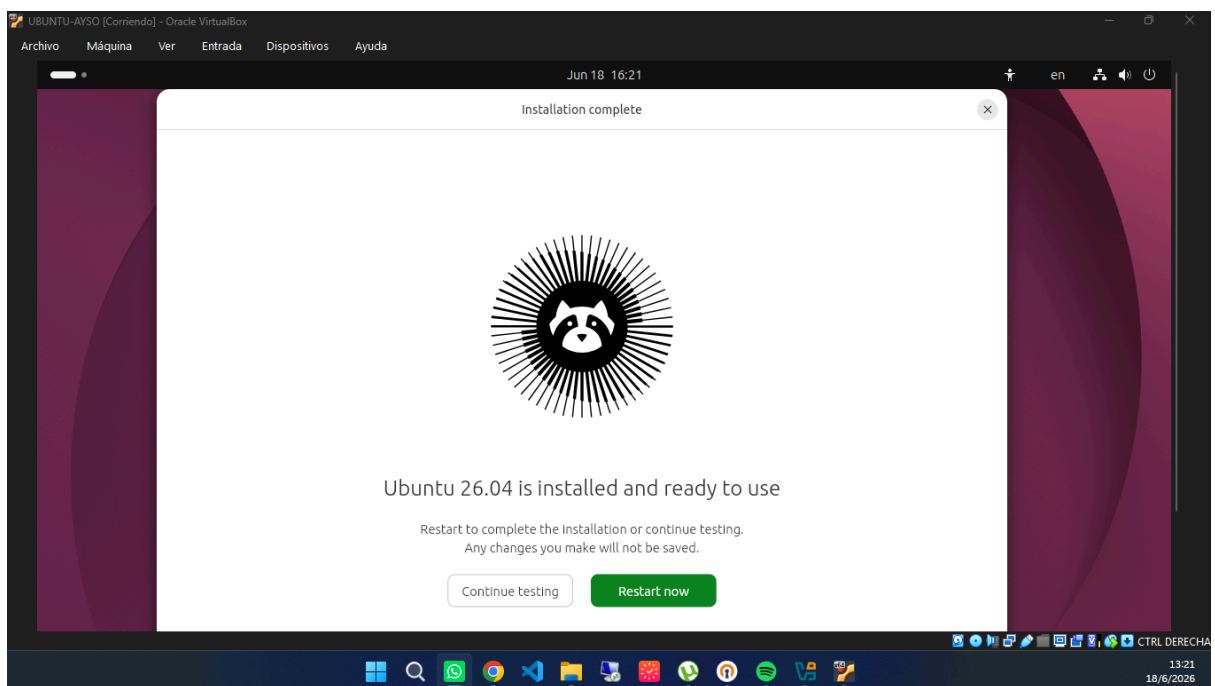


Figura 7. Proceso de Instalación de Ubuntu finalizado en la nueva máquina virtual.

4. Preparación del Entorno de Programación

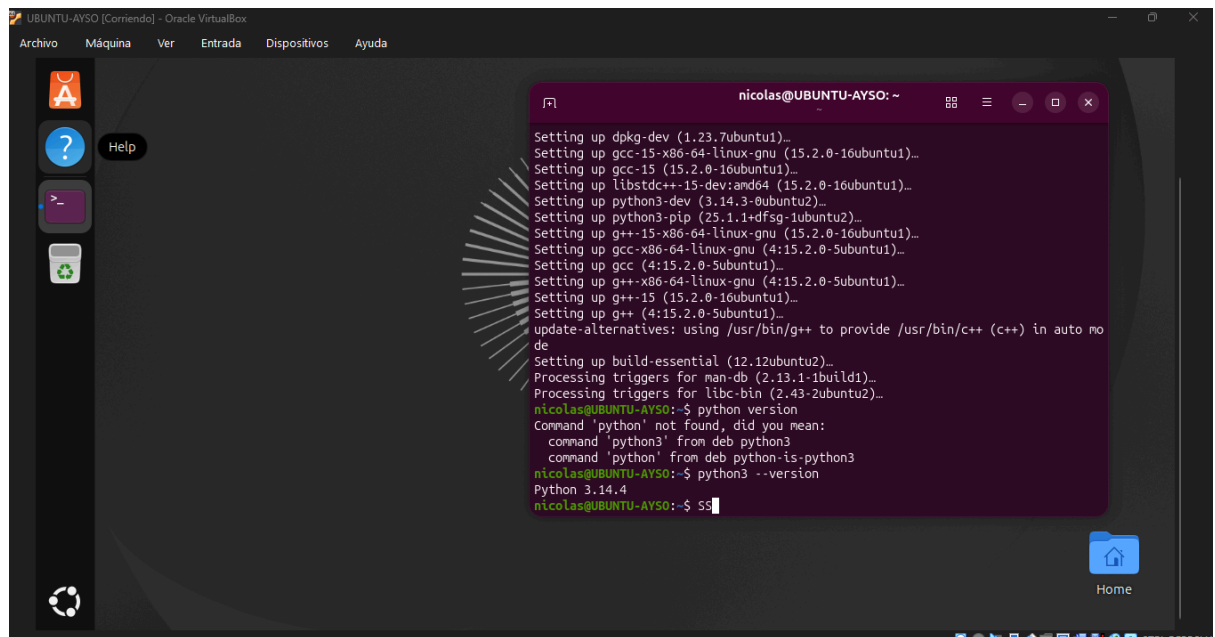
Una vez instalado el sistema operativo invitado, se accedió a la terminal de Linux con el propósito de instalar python a fin de preparar el entorno de ejecución de Python mediante los siguientes comandos:

Actualización de repositorios: `sudo apt update`.

Instalación del intérprete: `sudo apt install python3 python3-pip`.

Verificación: Se comprobó la versión instalada con `python3 --version`

En la **figura 8** se observa una captura , en la cual se verifica la versión del programa python instalado.



```

Setting up dpkg-dev (1.23.7ubuntu1)...
Setting up gcc-15-x86-64-linux-gnu (15.2.0-16ubuntu1)...
Setting up gcc-15 (15.2.0-16ubuntu1)...
Setting up libstdc++-15-dev:amd64 (15.2.0-16ubuntu1)...
Setting up python3-dev (3.14.3-0ubuntu2)...
Setting up python3-pip (25.1.1+dfsg-1ubuntu2)...
Setting up g++-15-x86-64-linux-gnu (15.2.0-16ubuntu1)...
Setting up gcc-x86-64-linux-gnu (4:15.2.0-5ubuntu1)...
Setting up gcc (4:15.2.0-5ubuntu1)...
Setting up g++-x86-64-linux-gnu (4:15.2.0-5ubuntu1)...
Setting up g++-15 (15.2.0-16ubuntu1)...
Setting up g++ (4:15.2.0-5ubuntu1)...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mode
Setting up build-essential (12.12ubuntu2)...
Processing triggers for man-db (2.13.1-1build1)...
Processing triggers for libc-bin (2.43-2ubuntu2)...
nicolas@UBUNTU-AYSO:~$ python version
Command 'python' not found, did you mean:
  command 'python3' from deb python3
  command 'python' from deb python-is-python3
nicolas@UBUNTU-AYSO:~$ python3 --version
Python 3.14.4
nicolas@UBUNTU-AYSO:~$ ss

```

Figura 8. Verificación de la instalación de Python.

5. Ejecución del Script y Control de Versiones

El objetivo final fue la ejecución del **script promedio.py**, el cual fue gestionado a través de un repositorio colaborativo en GitHub. La práctica concluyó con la descarga y ejecución exitosa del código dentro de la terminal de Ubuntu, validando así la funcionalidad del entorno virtualizado como un espacio de pruebas seguro e independiente del sistema operativo principal.

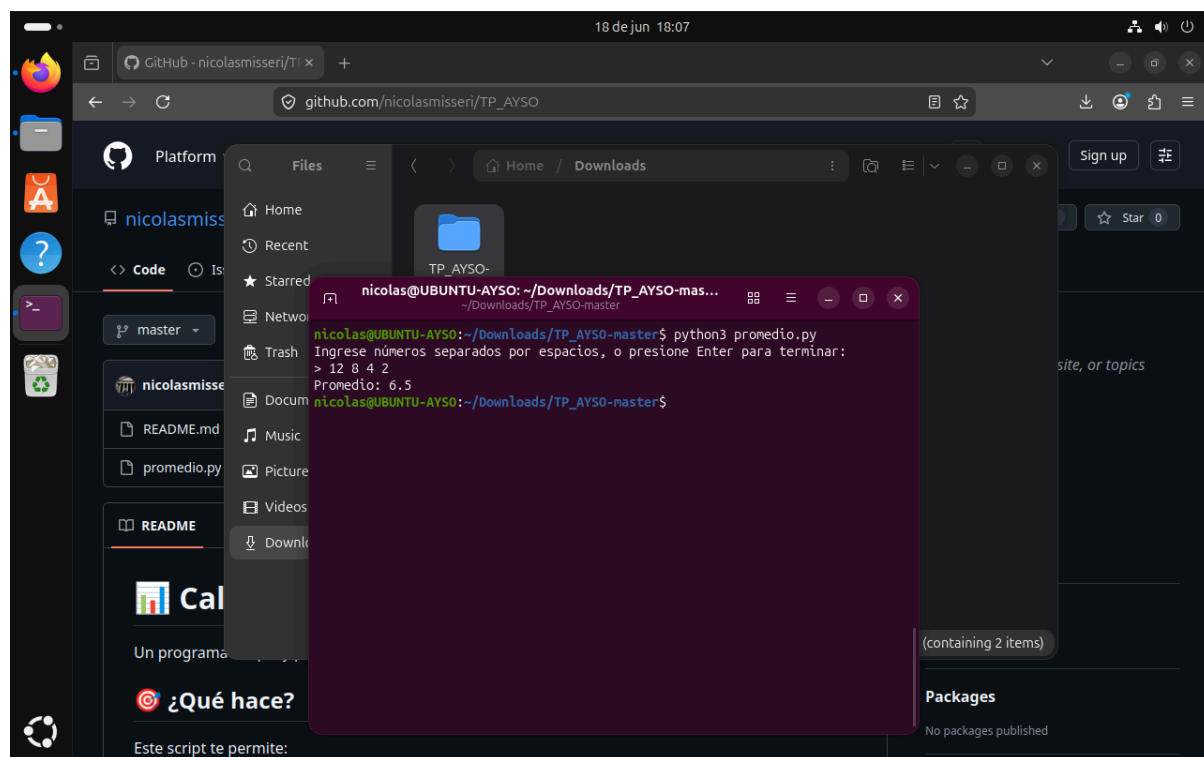


Figura 9. Captura con la ejecución del script promedio.py desde la máquina virtual.

4. Metodología Utilizada

El desarrollo del proyecto se estructuró a partir de una dinámica de trabajo colaborativo que integró la investigación técnica con la ejecución práctica. El proceso se dividió en las siguientes fases:

1. **Fase de Planificación e Intercambio de Ideas:** Previo a la ejecución técnica, ambos integrantes realizaron sesiones de intercambio de ideas para definir el alcance del proyecto y seleccionar las herramientas adecuadas (Ubuntu y Python) que mejor representaran los conceptos de la cátedra. En esta instancia se decidió utilizar un repositorio en GitHub para garantizar la consistencia del código fuente
2. **Fase de Implementación Técnica (Responsabilidad de Nicolás Misseri- Alumno 2):** Nicolas llevó a cabo la verificación de la virtualización en el hardware y la configuración del hipervisor Oracle VM VirtualBox. Asimismo, realizó la instalación del sistema

operativo Ubuntu 25.04 y la preparación del entorno de Python, generando el registro visual (capturas de pantalla) que sustenta el caso práctico.

3. **Fase de Integración y Redacción Técnica del proyecto** (Responsabilidad de Gerardo Ocampo- Alumno 1): Gerardo asumió la tarea de integrar las evidencias gráficas producidas con el sustento teórico de la asignatura. Se encargó de redactar el presente documento técnico, asegurando la coherencia entre el marco conceptual (hipervisores, sandbox, estándar FHS) y los pasos ejecutados en la práctica
4. **Herramientas Empleadas:** Se utilizó VirtualBox 7.2.10 como hipervisor de tipo 2, la terminal de Linux para la automatización de comandos, el intérprete de Python 3.14.4 para la lógica de programación y Git para el control de versiones

5. Resultados Obtenidos

Los resultados alcanzados al finalizar la práctica técnica fueron los siguientes:

- **Implementación Exitosa:** Se logró desplegar una máquina virtual funcional con Ubuntu, operando de manera fluida y con acceso a recursos emulados (CPU, RAM, Disco).
- **Aislamiento Comprobado:** Se validó que los procesos ejecutados dentro de la MV no interfirieron con las aplicaciones del sistema Windows, cumpliendo el principio de Sandbox.
- **Ejecución de Software:** El script de cálculo de promedios se ejecutó correctamente en la versión Python 3.14.4. Las pruebas de ingreso de datos por consola arrojaron resultados precisos (ej. promedio de 6.5 para la serie 12, 8, 4, 2), validando la operatividad del entorno.
- **Conectividad:** Se verificó la comunicación de red exitosa, permitiendo la interacción con repositorios externos y la descarga de paquetes mediante apt.

No se reportaron fallos críticos de hardware, aunque se observó que el rendimiento de la MV depende estrechamente de la cantidad de recursos remanentes en el sistema anfitrión.

6. Conclusiones

Se concluye que la virtualización es una herramienta estratégica que redefine la administración de sistemas modernos. La práctica permitió comprender cómo un hipervisor de tipo 2 abstrae la complejidad del hardware para ofrecer entornos flexibles y seguros.

Desde el punto de vista del aprendizaje, la práctica reforzó el conocimiento sobre la jerarquía de directorios en Linux bajo el estándar FHS, pilar fundamental para la administración de sistemas operativos libres. Asimismo, la resolución de la habilitación de la virtualización en la BIOS/UEFI representó un desafío técnico superado que fortaleció las capacidades de soporte del equipo

Como mejora futura, se propone explorar la virtualización ligera mediante contenedores (Docker), la cual, al compartir el kernel del host, podría ofrecer una mayor eficiencia en el consumo de RAM comparada con las máquinas virtuales tradicionales.

Finalmente, se destaca la importancia de la documentación técnica y el trabajo en equipo como competencias clave para el ejercicio profesional de la programación.

7. Bibliografía

- Oracle Corporation. (2024). Oracle VirtualBox. Fecha 20 de junio de 2026, de <https://www.virtualbox.org>.
- Ubuntu Documentation: Desktop Guide. Fecha 21 de Junio de 2026 de <https://help.ubuntu.com/>
- Universidad Tecnológica Nacional. (2026). Materiales de la Cátedra: Arquitectura y Sistemas Operativos [Módulo 7]

8. Anexos

Código Fuente (promedio.py): Disponible en el repositorio compartido: [nicolasmisseri/TP_AYSO](https://github.com/nicolasmisseri/TP_AYSO).

En el mismo el alumno 1 : Gerardo Ocampo es colaborador y el alumno 2: Nicolás Misseri es el titular de la cuenta GitHub.

Capturas de Pantalla: Evidencias de trabajo colaborativo

En las siguientes capturas se aportan evidencia del trabajo colaborativo; Commits de GitHub (Figura 10) e Historial de Versiones en Google Doc (Figura 11).

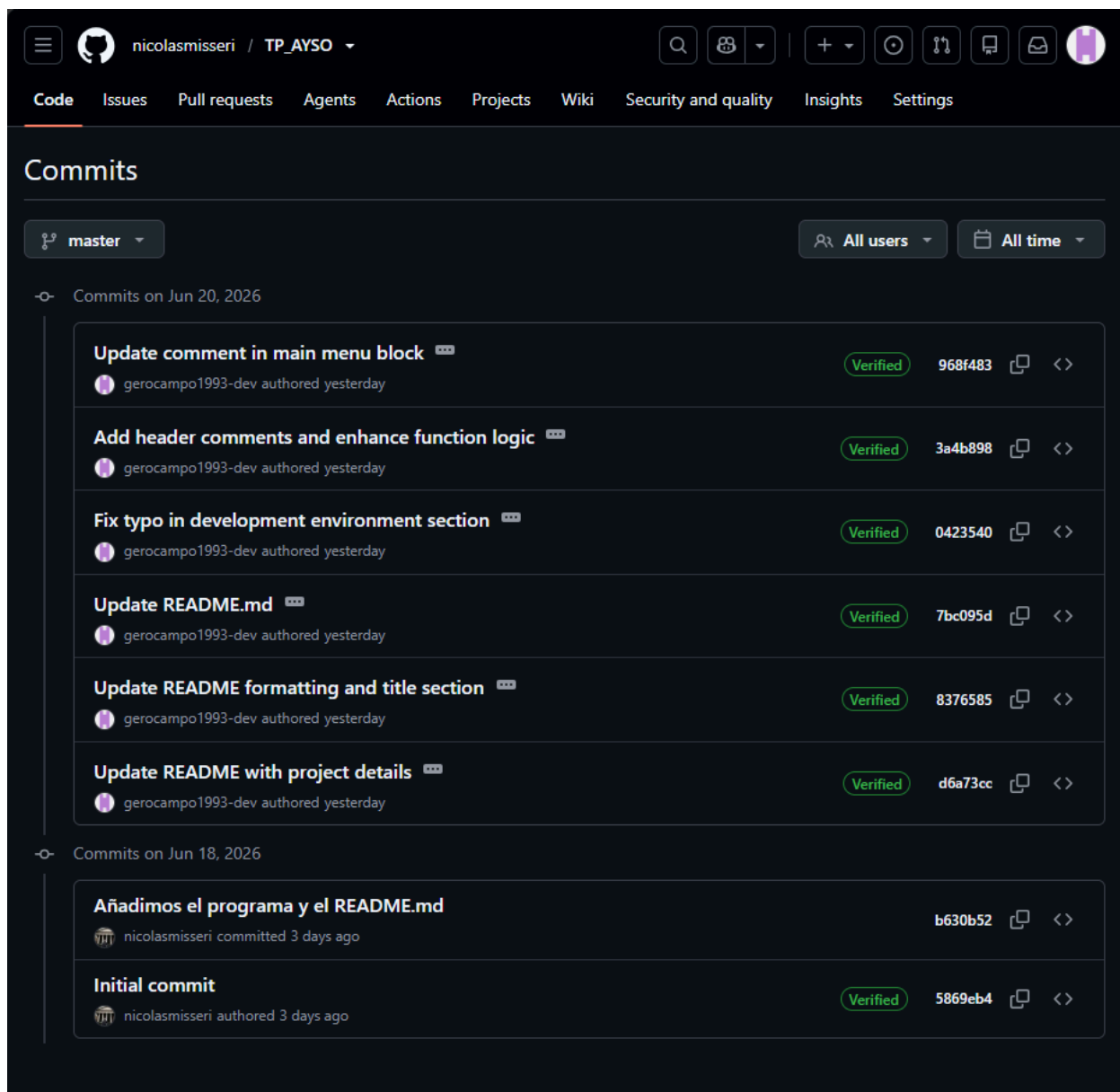


Figura 10. Capturas de commits de Github.

58 p.m.

Restablecer esta versión

Total: 5 cambios

Página 10 de 15

Tecnicatura Universitaria en Programación a Distancia - Arquitectura y Sistemas Operativos

Imagen ISO	Se utilizó la distribución Ubuntu Desktop (versión 25.04)
Memoria RAM	Se asignaron 3072 MB (aprox. 3 GB) para garantizar la fluidez del entorno gráfico

Memoria RAM: Se asignaron 3072 MB (aprox. 3 GB) para garantizar la fluidez del entorno gráfico

Procesador: Se destinaron 2 núcleos de CPU virtuales

Almacenamiento: Se configuró un disco duro virtual de 15 GB

Red: Se estableció una conexión en modo NAT para otorgar conectividad a Internet a la MV

Historial de versiones

Todas las versiones

Gerardo Ocampo

21 de junio, 4:57 p.m.

Gerardo Ocampo

21 de junio, 4:57 p.m.

Gerardo Ocampo

21 de junio, 4:56 p.m.

Gerardo Ocampo

21 de junio, 2:39 p.m.

Gerardo Ocampo

21 de junio, 9:48 a.m.

Gerardo Ocampo

Ayer

20 de junio, 9:56 p.m.

Gerardo Ocampo

20 de junio, 9:45 a.m.

Gerardo Ocampo

20 de junio, 8:45 a.m.

Gerardo Ocampo

Viernes

19 de junio, 9:33 p.m.

Gerardo Ocampo

Martes

16 de junio, 2:31 p.m.

Gerardo Ocampo

16 de junio, 12:30 p.m.

Nicolas Misseri

Figura 11. Capturas de Commits de Google Doc- Redacción del presente informe.